



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6

із дисципліни «Технології розробки програмного забезпечення»

**Тема: «ШАБЛони «Abstract Factory», «Factory Method», «Memento», «Observer»,
«Decorator»»**

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Завдання.....	3
Теоретичні відомості.....	3
Обрана тема.....	4
Клас <code>AudioStream</code>	5
Класи <code>HighQualityStream</code> , <code>LowQualityStream</code> та <code>MediumQualityStream</code>	5
Клас <code>AudioStreamFactory</code>	6
Використання <code>AudioStreamFactory</code>	6
Тестування <code>StreamingService</code> та <code>AudioStreamFactory</code>	7
Висновок.....	8

Тема: Шаблони «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Мета: Навчитися реалізовувати шаблони «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Завдання:

1. Реалізувати не менше 3-х класів відповідно до обраної теми.
2. Реалізувати один з розглянутих шаблонів за обраною темою.

Теоретичні відомості

Шаблони проєктування

Шаблон проєктування — це типовий підхід до вирішення поширених проблем, які виникають під час створення архітектури програмного забезпечення. Вони не є готовими бібліотеками чи функціями, які можна просто скопіювати в проєкт. Замість цього шаблон описує загальний принцип, який потребує адаптації до конкретних умов програми.

Найпростіші шаблони, які мають сенс лише в певній мові програмування, називають *ідіомами*. Найбільш універсальні — це *архітектурні шаблони*, які можна застосувати в будь-якій мові програмування. Їх використовують для створення структури програми в цілому, а не окремих її частин.

Класифікація шаблонів

Шаблони можна поділити на три основні групи:

1. **Породжуючі:** допомагають створювати об'єкти, зменшуючи кількість залежностей у програмі.
2. **Структурні:** визначають, як об'єкти можуть взаємодіяти між собою.
3. **Поведінкові:** оптимізують обмін даними та комунікацію між об'єктами.

Abstract Factory

Патерн Abstract Factory забезпечує інтерфейс для створення сімейства пов'язаних або залежних об'єктів без зазначення їх конкретних класів. Це досягається через визначення

абстрактного інтерфейсу фабрики, а конкретні фабрики реалізують його для створення продуктів. Підвищує узгодженість об'єктів і спрощує розширення сімейств продуктів.

Factory Method

Factory Method делегує створення об'єктів до підкласів, дозволяючи їм визначити тип об'єкта, який буде створений. Замість прямого створення об'єктів (через new), клієнт використовує метод фабрики. Це сприяє поліморфізму та зменшує залежність від конкретних класів.

Memento

Патерн Memento дозволяє зберігати та відновлювати попередній стан об'єкта, не розкриваючи деталей його реалізації. Він включає три компоненти: Originator (створює та відновлює стан), Memento (зберігає стан), і Caretaker (керує збереженням). Часто використовується для реалізації функцій "відміна/повтор".

Observer

Observer встановлює залежність "один-до-багатьох" між об'єктами, де один об'єкт (Subject) повідомляє залежні об'єкти (Observers) про зміну свого стану. Це дозволяє динамічно оновлювати дані в залежних об'єктах без жорсткої прив'язки до них, підвищуючи модульність системи.

Decorator

Патерн Decorator динамічно додає нову поведінку об'єкту, обгортаючи його в інший об'єкт. Він використовує композицію замість наслідування, дозволяючи створювати багато комбінацій поведінки за допомогою окремих декораторів. Це забезпечує гнучкість у модифікації функціоналу без змін базового класу.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Під час розробки програми я реалізував шаблон "Factory method", оскільки у проєкті "Online radio station" має бути реалізована функція мовлення в потоковому режимі для різних бітрейтів, що і дозволяє реалізувати цей метод програмування. Від

абстрактного класу `AudioStream` створенні класи `HighQualityStream`, `LowQualityStream` та `MediumQualityStream` для різних бітрейтів.

Абстрактний клас `AudioStream`:

```
package com.OnlineRadio.OnlineRadioStation.models.streaming;

public abstract class AudioStream {
    protected int bitrate;

    public AudioStream(int bitrate) {
        this.bitrate = bitrate;
    }

    public abstract String getStreamQuality();

    public int getBitrate() {
        return bitrate;
    }

    public void startStream() {
        System.out.println("Streaming started at " + bitrate + " kbps.");
    }
}
```

Рисунок 1. Абстрактний клас `AudioStream`

Класи `HighQualityStream`, `LowQualityStream` та `MediumQualityStream`

```
package com.OnlineRadio.OnlineRadioStation.models.streaming;

public class HighQualityStream extends AudioStream {
    public HighQualityStream() {
        super(256);
    }

    @Override
    public String getStreamQuality() {
        return "High Quality (256 kbps)";
    }
}
```

Рисунок 2. Клас `HighQualityStream`

```
package com.OnlineRadio.OnlineRadioStation.models.streaming;

public class LowQualityStream extends AudioStream {
    public LowQualityStream() {
        super(64);
    }

    @Override
    public String getStreamQuality() {
        return "Low Quality (64 kbps)";
    }
}
```

Рисунок 3. Клас `LowQualityStream`

```

package com.OnlineRadio.OnlineRadioStation.models.streaming;

public class MediumQualityStream extends AudioStream {
    public MediumQualityStream() {
        super(128); // 128 kbps
    }

    @Override
    public String getStreamQuality() {
        return "Medium Quality (128 kbps)";
    }
}

```

Рисунок 4. Клас LowQualityStream

Клас AudioStreamFactory, що реалізує основну логіку паттерну:

```

package com.OnlineRadio.OnlineRadioStation.factories;

import com.OnlineRadio.OnlineRadioStation.models.streaming.*;

public class AudioStreamFactory {

    public static AudioStream createStream(String quality) {
        switch (quality.toLowerCase()) {
            case "low":
                return new LowQualityStream();
            case "medium":
                return new MediumQualityStream();
            case "high":
                return new HighQualityStream();
            default:
                throw new IllegalArgumentException("Unknown stream
quality: " + quality);
        }
    }
}

```

Рисунок 5. Клас AudioStreamFactory, що реалізує основну логіку паттерну

Використання AudioStreamFactory у StreamingService

```

package com.OnlineRadio.OnlineRadioStation.services;

import com.OnlineRadio.OnlineRadioStation.models.streaming.AudioStream;
import com.OnlineRadio.OnlineRadioStation.factories.AudioStreamFactory;
import org.springframework.stereotype.Service;

@Service
public class StreamingService {

    private AudioStream currentStream;

    public void startStreaming(String quality) {
        currentStream = AudioStreamFactory.createStream(quality);
        System.out.println("Starting " + currentStream.getStreamQuality()
+ " stream...");
        currentStream.startStream();
    }
}

```

```

        public AudioStream getCurrentStream() {
            return currentStream;
        }
    }
}

```

Рисунок 6. Використання AudioStreamFactory у StreamingService

Тестування StreamingService та AudioStreamFactory

```

package com.OnlineRadio.OnlineRadioStation.services;

import com.OnlineRadio.OnlineRadioStation.models.streaming.AudioStream;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class StreamingServiceTest {

    @Test
    void testStartLowQualityStreaming() {
        StreamingService streamingService = new StreamingService();
        streamingService.startStreaming("low");

        AudioStream stream = streamingService.getCurrentStream();
        assertNotNull(stream);
        assertEquals(64, stream.getBitrate());
        assertEquals("Low Quality (64 kbps)", stream.getStreamQuality());
    }

    @Test
    void testStartMediumQualityStreaming() {
        StreamingService streamingService = new StreamingService();
        streamingService.startStreaming("medium");

        AudioStream stream = streamingService.getCurrentStream();
        assertNotNull(stream);
        assertEquals(128, stream.getBitrate());
        assertEquals("Medium Quality (128 kbps)",
stream.getStreamQuality());
    }

    @Test
    void testStartHighQualityStreaming() {
        StreamingService streamingService = new StreamingService();
        streamingService.startStreaming("high");

        AudioStream stream = streamingService.getCurrentStream();
        assertNotNull(stream);
        assertEquals(256, stream.getBitrate());
        assertEquals("High Quality (256 kbps)",
stream.getStreamQuality());
    }
}

```

Рисунок 7. Тестування StreamingService та AudioStreamFactory

```
"C:\Program Files\Java\jdk-17.0.4.1\bin\java.exe" ...  
Starting High Quality (256 kbps) stream...  
Streaming started at 256 kbps.  
Starting Low Quality (64 kbps) stream...  
Streaming started at 64 kbps.  
Starting Medium Quality (128 kbps) stream...  
Streaming started at 128 kbps.
```

Рисунок 8. Результати тестування StreamingService та AudioStreamFactory

Висновок: На цій лаб. роботі я зрозумів суть шаблонів шаблони «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator», реалізував один з них (Factory Method) для свого проекту “Online radio station”, адже необхідно було реалізувати потокове мовлення з підтримкою різних бітрейтів. Для цього від абстрактного класу `AudioStream` були створені класи `HighQualityStream`, `LowQualityStream` та `MediumQualityStream`, кожен з яких відповідає певному бітрейту.